

FractionalDiffEq.jl: Efficient Fractional Differential Equations Solving Manual

June 9, 2026

[erikqqy123@gmail.com]Qingyu Qu

Contents

1	Introduction	2
2	Installation	2
3	Available Solvers	3
3.1	Fractional Ordinary Differential Equations	3
3.2	Multi-Term Fractional Differential Equations	3
3.3	Fractional Delay Differential Equations	4
3.4	Solver Selection Recommendations	4
4	Solving Fractional Differential Equations	4
4.1	Solve fractional order Chua chaotic system	5
4.2	Solve Bagley-Torvik equation	6
5	Special Function treatment	7
6	Manipulate Solution Output	8
6.1	Accessing Solution Values	8
6.2	Controlling Output Locations	8
6.3	Interpolating Solutions	9
6.4	Converting Solutions to Arrays	9
7	Visualization Solutions	9
7.1	Time-Series Visualization	9
7.2	Phase Portraits	10
7.3	Three-Dimensional Chaotic Attractors	10
8	Application: Parameter estimation	10
9	Conclusion	11

1 Introduction

Fractional differential equations (FDEs) have emerged as a powerful mathematical framework for modeling systems with memory and hereditary effects. Unlike classical ordinary differential equations, where the future evolution of a system depends only on its current state, fractional differential equations incorporate the entire history of the state variables through nonlocal fractional-order operators. This property makes fractional-order models particularly suitable for describing complex phenomena in viscoelastic materials, anomalous diffusion, dielectric polarization, biological systems, control engineering, signal processing, and many other scientific and engineering applications.

FractionalDiffEq.jl is a high-performance Julia package for numerically solving fractional differential equations within the SciML ecosystem. The package provides a unified and extensible framework for constructing, solving, and analyzing a wide range of fractional-order dynamical systems. Inspired by the design principles of DifferentialEquations.jl [Rackauckas and Nie(2017)], FractionalDiffEq.jl adopts the same problem-oriented workflow:

1. Define a mathematical problem using an appropriate problem type;
2. Select a numerical algorithm suitable for the problem;
3. Solve the problem through a unified `solve` interface;
4. Analyze, visualize, or post-process the solution.

This design allows users familiar with the SciML ecosystem to seamlessly transition from classical differential equations to fractional-order models while retaining access to the broader Julia scientific computing infrastructure.

Currently, FractionalDiffEq.jl supports several important classes of fractional differential equations, including:

- Fractional ordinary differential equations (FODEs);
- Multi-term fractional differential equations;
- Fractional delay differential equations (FDDEs);
- Bagley–Torvik equations and related special formulations.

The package provides a collection of state-of-the-art numerical methods specifically designed for fractional-order systems, ranging from predictor–corrector methods and fractional linear multistep schemes to fractional backward differentiation formulas. Furthermore, the package follows SciML conventions for solution handling, enabling straightforward integration with visualization, optimization, sensitivity analysis, uncertainty quantification, and machine learning workflows.

The objective of this manual is to introduce the core functionality of FractionalDiffEq.jl and provide practical examples illustrating how fractional differential equations can be defined, solved, and analyzed in Julia.

2 Installation

FractionalDiffEq.jl is registered in the Julia General Registry and can be installed directly using Julia’s package manager.

Open a Julia session and execute:

```
1 using Pkg
2 Pkg.add("FractionalDiffEq")
```

After installation, load the package with:

```
1 using FractionalDiffEq
```

To verify that the installation has completed successfully, users may inspect the package version:

```
1 using Pkg
2 Pkg.status("FractionalDiffEq")
```

FractionalDiffEq.jl is developed entirely in Julia and is compatible with modern versions of the Julia language. Since the package follows SciML conventions, it can be used together with many other packages in the Julia scientific computing ecosystem, including:

- `CairoMakie.jl` for visualization;
- `Optimization.jl` for parameter estimation;
- `SciMLSensitivity.jl` for sensitivity analysis;
- `GlobalSensitivity.jl` for global sensitivity analysis;
- `QuasiMonteCarlo.jl` for uncertainty quantification;
- `ModelingToolkit.jl` for symbolic model construction.

Additional package-specific examples and tutorials are available in the online documentation and repository examples directory.

3 Available Solvers

Different classes of fractional differential equations often require different numerical techniques. To accommodate a broad range of applications, FractionalDiffEq.jl implements several numerical methods specifically designed for fractional-order systems. These algorithms differ in stability properties, computational complexity, memory requirements, and suitability for particular classes of problems.

3.1 Fractional Ordinary Differential Equations

For standard Caputo fractional ordinary differential equations, FractionalDiffEq.jl provides several predictor–corrector and multistep methods:

- `PECE()` — Predictor–Evaluate–Correct–Evaluate method based on the Adams–Bashforth–Moulton formulation;
- `PIEX()` — Predictor–Integral Explicit method for non-stiff fractional systems;
- `BDF()` — Fractional backward differentiation formula suitable for stiff problems.

These methods can be applied through the unified `solve` interface:

```
1 sol = solve(prob, PECE(), dt=0.01)
2 sol = solve(prob, PIEX(), dt=0.01)
3 sol = solve(prob, BDF(), dt=0.01)
```

3.2 Multi-Term Fractional Differential Equations

For equations involving multiple fractional derivatives of different orders, FractionalDiffEq.jl provides dedicated numerical methods:

- `MTPECE()` — Multi-Term Predictor–Corrector method;
- `GL()` — Grünwald–Letnikov based discretization scheme.

These methods are particularly useful for viscoelastic models, constitutive equations, and Bagley–Torvik-type problems.

```
1 sol = solve(prob, MTPECE(), dt=0.01)
2 sol = solve(prob, GL(),      dt=0.01)
```

3.3 Fractional Delay Differential Equations

Fractional delay differential equations combine memory effects with explicit time delays and often require specialized treatment. `FractionalDiffEq.jl` provides numerical methods designed to handle both fractional history terms and delayed state evaluations while preserving accuracy and stability.

3.4 Solver Selection Recommendations

As a general guideline:

- Use `PECE()` for general-purpose fractional ordinary differential equations;
- Use `PIEX()` when computational efficiency is prioritized for non-stiff systems;
- Use `BDF()` for stiff fractional systems;
- Use `MTPECE()` for multi-term fractional differential equations;
- Use `GL()` when a Grünwald–Letnikov formulation is desired.

Because all numerical methods share the same `solve` interface, switching between algorithms typically requires changing only the solver object, enabling convenient benchmarking and method comparison for a given problem.

4 Solving Fractional Differential Equations

`FractionalDiffEq.jl` adopts the same high-level workflow used throughout the SciML ecosystem [Rackauckas and Nie(2017)]: users first formulate a mathematical problem by constructing an appropriate problem object and subsequently solve it using a numerical algorithm through the unified `solve` interface. This design philosophy separates the mathematical description of a model from the numerical method used to solve it, allowing users to easily experiment with different algorithms without modifying the underlying model definition.

In a typical workflow, users specify the governing fractional differential equation together with the corresponding initial conditions, fractional orders, simulation time span, and model parameters. Once the problem has been defined, a suitable numerical solver can be selected and invoked through a single call to `solve`. This abstraction provides a consistent and intuitive interface that closely follows the conventions established by the SciML differential equations ecosystem, making it straightforward for users familiar with `OrdinaryDiffEq.jl`, `BoundaryValueDiffEq.jl`, or other SciML packages to transition to fractional-order modeling.

Compared with classical ordinary differential equations, fractional differential equations introduce nonlocal operators that inherently possess memory effects. In contrast to integer-order derivatives, which depend only on local information at the current time instant, fractional derivatives incorporate the entire historical evolution of the system through convolution-type operators. As a consequence, the future state of a fractional-order system depends not only on its current state but also on its past trajectory. This property makes fractional differential equations particularly suitable for modeling phenomena exhibiting hereditary characteristics, long-range temporal dependence, viscoelastic behavior, anomalous diffusion, dielectric polarization, biological memory processes, and many other complex dynamical systems.

Because different classes of fractional differential equations exhibit distinct mathematical structures and numerical challenges, `FractionalDiffEq.jl` provides specialized problem abstractions and solver implementations tailored to various categories of fractional-order models. Currently supported problem types include:

- **Fractional Ordinary Differential Equations (FODEs):** nonlinear or linear systems containing one or more Caputo fractional derivatives, representing the most common class of fractional dynamical systems;
- **Multi-Term Fractional Differential Equations:** differential equations involving multiple fractional derivatives of different orders acting on the same unknown function, frequently encountered in viscoelasticity, control theory, and fractional constitutive modeling.

Each problem type is associated with numerical methods specifically designed for its underlying mathematical formulation. By encapsulating these distinctions within dedicated problem definitions, FractionalDiffEq.jl enables users to focus on model development while automatically leveraging algorithms appropriate for the selected class of fractional differential equations. Furthermore, the modular architecture of FractionalDiffEq.jl facilitates future extensions, allowing additional classes of fractional differential equations and numerical algorithms to be incorporated while preserving the same unified user interface.

The basic structure of FractionalDiffEq.jl when solving fractional differential equations with different types of problem is shown in Figure 1:

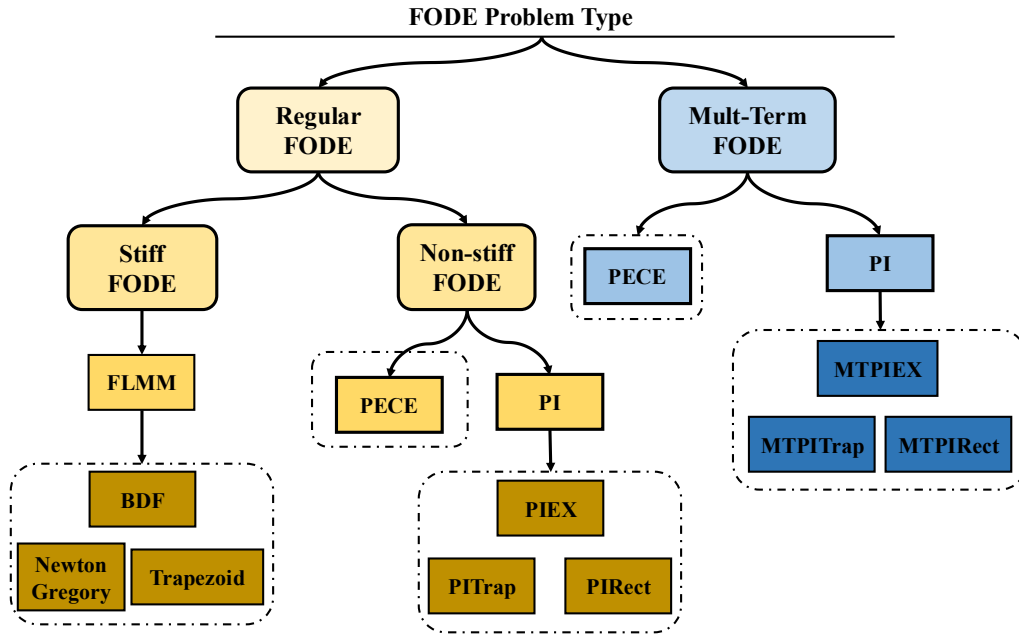


Figure 1: **Schematic of FractionalDiffEq.jl** : FractionalDiffEq.jl has a modular architecture where different algorithms should be chosen concerning the problem properties according to different kinds of fractional ordinary differential equations.

In the following examples, we demonstrate how to solve representative fractional-order systems using FractionalDiffEq.jl.

4.1 Solve fractional order Chua chaotic system

Classical Chua's circuit is a simple model that exhibits nonlinear dynamical phenomena including bifurcation and chaos with appropriate configuration. According to I. Petráš et al. [Petráš(2011)], when the capacitor and inductor in Chua's system are replaced by the new linear capacitor model and inductor model which is based on Curie's empirical law of 1889 [Westerlund and Ekstam(2002)], the general input voltage and current would behave

a fractional-order derivative relationship as:

$$I(t) = C \frac{d^\alpha V(t)}{dt^\alpha} \equiv C_0 D_t^\alpha V(t), \quad V(t) = L \frac{d^\alpha I(t)}{dt^\alpha} \equiv L_0 D_t^\alpha I(t) \quad (1)$$

where C is the capacitance of the real capacitor and L is the inductance of the inductor. The Caputo derivative here can better describe the memory effects of the charging and discharging process of capacitor, and the energizing and de-energizing process of inductor [Westerlund(1991)]. Hence we have the fractional-order Chua chaotic system as:

$$\begin{cases} {}_0^C D_t^{\alpha_1} u_1(t) = a(u_2(t) - u_1(t) - f(u_1)) \\ {}_0^C D_t^{\alpha_2} u_2(t) = u_1(t) - u_2(t) + u_3(t) \\ {}_0^C D_t^{\alpha_3} u_3(t) = -bu_2(t) - cu_3(t) \end{cases} \quad (2)$$

To utilize FractionalDiffEq.jl solve the fractional Chua system, we simply need to follow the use the "define the problem then solve the problem" convention. To maintain consistency with the interface design of [Rackauckas and Nie(2017)] and ensure seamless integration within the SciML differential equations solvers ecosystem, FractionalDiffEq.jl adopts a similar formulation style for defining fractional ordinary differential equations. This design choice facilitates user familiarity and interoperability with existing numerical solvers, thereby enhancing usability and accessibility for researchers and practitioners. The primary distinction, however, lies in the explicit specification of the fractional order in the Caputo sense, which is a required parameter in FractionalDiffEq.jl during problem construction.

```

1 using FractionalDiffEq
2 function chua!(du, x, p, t)
3     a, b, c, m0, m1 = p
4     du[1] =
5         ↪ a*(x[2]-x[1]-(m1*x[1]+m0*(abs(x[1]+1)-abs(x[1]-1))))
6     du[2] = x[1]-x[2]+x[3]
7     du[3] = -b*x[2]-c*x[3]
8 end
9 alpha = [0.93, 0.99, 0.92];
10 u0 = [0.2; -0.1; 0.1];
11 tspan = (0, 100);
12 p = [10.725, 10.593, 0.268, -0.1927, -0.7872]
13 prob = FODEProblem(chua!, alpha, u0, tspan, p)
14 sol = solve(prob, BDF(), dt=0.01)

```

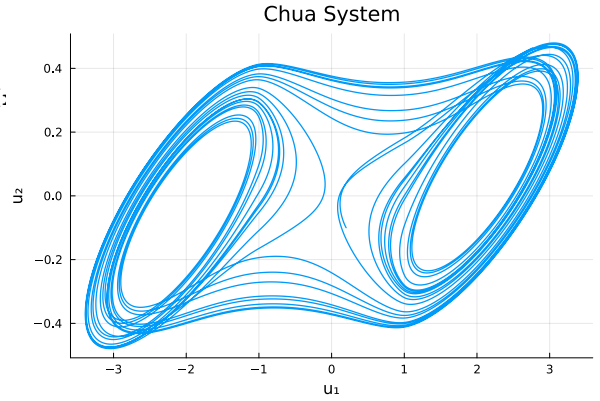


Figure 2: FractionalDiffEq.jl allows seamless solvers switching between different FODE numerical algorithms for FODE problems. We can choose different solvers according to the commensurate order of the FODE system from available solvers in FractionalDiffEq.jl.

4.2 Solve Bagley-Torvik equation

The Bagley-Torvik equation arises in the modeling of the motion of a rigid plate immersed in a Newtonian fluid [Podlubny(1998), Diethelm and Ford(2002)], which describes the relationship between the stress and velocity for the geometry and loading can be described in a fractional-order manner:

$$\sigma(t, z) = \sqrt{\mu \rho_0} D_t^{1/2} v(t, z) \quad (3)$$

where ρ is the fluid density, μ is the viscosity of the fluid and $.$ So the dominant fractional differential equation can be formulated:

$$\begin{aligned} Au''(t) + B_0^C D_t^{3/2} u(t) + Cu(t) &= f(t) \\ f(t) &= \begin{cases} 0, & t > 1 \\ 8, & 0 \leq t \leq 1 \end{cases} \end{aligned} \quad (4)$$

where $A \neq 0$ and $B, C \in \mathcal{R}$, the initial conditions are chosen as $u(0) = u'(0) = 1$ since the displacement and velocity of the plate-fluid system must be in equilibrium for the fractional derivative to be applicable [Podlubny(1998)]. The fractional derivative $D_t^{3/2}u(t)$ introduced in this equation is used to interpret the relationship of stress at a given point at any time is dependent on the time history of the velocity profile at the point.

```

1 using FractionalDiffEq
2
3 tspan = (0.0, 20.0)
4 rhs(u, p, t) = ifelse(t > 1, 0, 8)
5 u0 = [0.0, 0.0]
6 prob = MultiTermsFODEProblem([1, 1/2,
  ↪ 1/2], [2, 1.5, 0], rhs, u0, tspan)
7 sol = solve(prob, MTPECE(), dt=0.01)

```

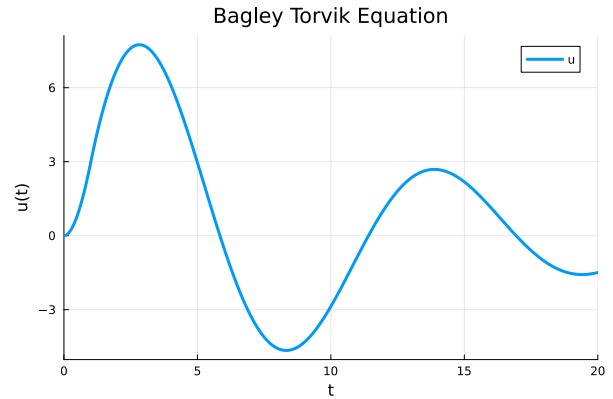


Figure 3: FractionalDiffEq.jl allows seamless solvers switching between different numerical algorithms for multi-term FODE problems. We can choose any available multi-term FODE solver during calling `solve` from available solvers in FractionalDiffEq.jl.

Because FractionalDiffEq.jl adopts a unified solver interface, switching between different multi-term fractional solvers is straightforward:

```

1 sol = solve(prob, MTPECE(), dt=0.01)
2 sol = solve(prob, GL(),      dt=0.01)

```

This design allows users to compare numerical methods and select the most suitable algorithm according to the problem characteristics.

5 Special Function treatment

Fractional differential equations frequently involve special functions that naturally arise in the analytical representation of fractional-order systems. Among these, the Mittag-Leffler function plays a particularly important role, serving as the fractional analogue of the exponential function in classical differential equations. Other special functions, such as the Gamma function, Beta function, and generalized hypergeometric functions, also appear in exact solutions, stability analysis, and parameter estimation problems.

To facilitate fractional-order modeling workflows, FractionalDiffEq.jl provides seamless access to commonly used special functions through its integration with Julia’s scientific computing ecosystem. Users may directly utilize functions from the Julia package ecosystem when defining fractional differential equations, constructing forcing terms, or evaluating analytical reference solutions.

For example, the Mittag-Leffler function can be used to construct an analytical solution for validating numerical solvers:

```

1 using FractionalDiffEq
2 using SpecialFunctions
3
4 alpha = 0.8
5 t = 0:0.01:10
6
7 u_exact = [mittag_leffler(alpha, -ti^alpha) for ti in t]

```

Special functions can also be incorporated directly into the right-hand side of a fractional differential equation:

```
1 using FractionalDiffEq
2 using SpecialFunctions
3
4 function f(u, p, t)
5     return gamma(1.5) * exp(-t)
6 end
```

Since FractionalDiffEq.jl is fully compatible with Julia’s multiple-dispatch paradigm, any special function that operates on standard numerical types can generally be used without modification. This flexibility enables users to combine fractional differential equation models with a broad range of special-function libraries available in the Julia ecosystem.

For computationally intensive applications, users are encouraged to precompute frequently evaluated special functions whenever possible, thereby reducing repeated evaluations during numerical integration and improving overall simulation performance.

6 Manipulate Solution Output

All numerical algorithms in FractionalDiffEq.jl return a solution object that follows the common SciML solution interface. This unified design enables users to access solution values, interpolate states, convert solutions to arrays, and seamlessly integrate simulation results with downstream analysis tools.

6.1 Accessing Solution Values

After solving a fractional differential equation, the solution object stores both the time points and the corresponding numerical approximations:

```
1 sol = solve(prob, BDF(), dt = 0.01)
2
3 sol.t      # Time points
4 sol.u      # Solution values
```

Individual state variables can be extracted using standard Julia operations:

```
1 u1 = first(sol.u)
2 u2 = getindex(sol.u, 2)
3 u3 = getindex(sol.u, 3)
```

This representation is particularly convenient for post-processing and visualization.

6.2 Controlling Output Locations

By default, numerical algorithms determine suitable output locations according to the integration procedure. Users may explicitly specify desired output times using the **saveat** keyword:

```
1 sol = solve(prob, BDF(), dt = 0.01, saveat = 0:0.1:100)
```

This functionality is useful when comparing multiple simulations, performing uncertainty quantification, or preparing data for machine learning applications that require a common temporal grid.

6.3 Interpolating Solutions

Many applications require evaluating the solution at time points that do not coincide with the solver output grid. The solution object provides interpolation capabilities through function-call syntax:

```
1 u = sol(3.14159)
```

For systems of equations, the interpolated state vector can be accessed component-wise:

```
1 u = sol(5.0)
2
3 println(u[1])
4 println(u[2])
```

6.4 Converting Solutions to Arrays

For numerical analysis and machine learning workflows, it is often convenient to convert solution objects into matrix form:

```
1 using RecursiveArrayTools
2
3 A = Array(sol)
```

The resulting matrix contains the numerical solution in a format compatible with most Julia scientific computing packages.

7 Visualization Solutions

Visualization plays an important role in understanding the qualitative behavior of fractional-order dynamical systems. Since FractionalDiffEq.jl adopts the standard SciML solution interface, numerical solutions can be visualized using any plotting package in the Julia ecosystem.

In this manual we employ CairoMakie.jl for demonstration purposes.

7.1 Time-Series Visualization

The simplest form of visualization is plotting the evolution of a state variable with respect to time:

```
1 using CairoMakie
2 fig = Figure()
3 ax = Axis(fig[1, 1],
4 xlabel = "Time",
5 ylabel = "State")
6
7 lines!(ax, sol.t, first.(sol.u))
8 fig
```

This representation is useful for studying transient dynamics, oscillatory behavior, and long-term stability.

7.2 Phase Portraits

For multi-dimensional systems, phase portraits often reveal important dynamical structures:

```
1 u1 = first(sol.u)
2 u2 = getindex(sol.u, 2)
3 fig = Figure()
4 ax = Axis(fig[1, 1],
5 xlabel = L"u_1",
6 ylabel = L"u_2")
7 lines!(ax, u1, u2)
8 fig
```

Phase portraits are frequently used to analyze attractors, bifurcations, and nonlinear dynamics in fractional-order systems.

7.3 Three-Dimensional Chaotic Attractors

For chaotic systems such as the fractional-order Chua circuit, three-dimensional trajectories provide additional insight into the underlying dynamics:

```
1 u1 = first(sol.u)
2 u2 = getindex(sol.u, 2)
3 u3 = getindex(sol.u, 3)
4 fig = Figure()
5 ax = Axis3(fig[1,1])
6 lines!(ax, u1, u2, u3)
7 fig
```

Such visualizations are commonly employed when studying fractional-order chaotic systems and nonlinear oscillators.

8 Application: Parameter estimation

A common application of fractional differential equations is the identification of unknown model parameters from experimental observations. Owing to the memory effects introduced by fractional derivatives, fractional-order models often provide significantly improved fitting accuracy when compared with classical integer-order models.

Because FractionalDiffEq.jl follows SciML conventions, it can be naturally combined with the optimization and parameter estimation tools available in the Julia ecosystem.

Suppose that experimental observations are available at discrete time points. A loss function can be constructed by comparing the numerical solution with the observed data:

```
1 function loss(p)
2   prob = remake(prob_template; p = p)
3   sol = solve(prob, BDF(), dt = 0.01, saveat = observation_times)
4
5   return sum(abs2, Array(sol)[1, :] .- observations)
6 end
```

The parameter estimation problem can then be formulated as a standard optimization problem:

```

1 using Optimization
2 using OptimizationOptimJL
3
4 optf = OptimizationFunction((x, p) -> loss(x), Optimization.AutoFiniteDiff())
5
6 optprob = OptimizationProblem(optf, initial_guess)
7
8 result = solve(optprob, BFGS())

```

The optimized parameter vector is stored in:

```

1 result.u

```

Fractional-order parameter estimation has been successfully applied in numerous fields including viscoelasticity, battery modeling, anomalous diffusion, biological systems, epidemiology, and fermentation kinetics.

Since FractionalDiffEq.jl is compatible with the broader SciML ecosystem, users may further leverage advanced tools such as `Optimization.jl`, `SciMLSensitivity.jl`, `GlobalSensitivity.jl`, and `DiffEqParamEstim.jl` to perform large-scale inverse problems and uncertainty-aware parameter estimation.

9 Conclusion

FractionalDiffEq.jl provides a comprehensive and high-performance framework for modeling and simulating fractional differential equations within the Julia programming language. By adopting the design philosophy and interfaces of the SciML ecosystem, the package enables users to formulate fractional-order models using intuitive problem abstractions while retaining access to a broad collection of numerical algorithms and scientific computing tools.

Throughout this manual, we have demonstrated how to construct fractional differential equation problems, select appropriate numerical solvers, manipulate solution outputs, visualize simulation results, and perform parameter estimation tasks. The unified `solve` interface allows users to easily switch between numerical algorithms, facilitating method comparison and accelerating model development.

Beyond numerical integration, FractionalDiffEq.jl integrates naturally with the wider Julia scientific computing ecosystem, making it possible to combine fractional-order modeling with optimization, sensitivity analysis, uncertainty quantification, machine learning, and symbolic computation workflows. This interoperability significantly lowers the barrier to applying fractional differential equations in real-world scientific and engineering applications.

As research on fractional calculus continues to expand, FractionalDiffEq.jl aims to provide a robust, extensible, and user-friendly platform for fractional-order modeling, enabling researchers and practitioners to efficiently investigate complex dynamical systems exhibiting memory and hereditary effects.

References

- [Diethelm and Ford(2002)] Kai Diethelm and J Ford. 2002. Numerical solution of the Bagley-Torvik equation. *BIT numerical mathematics* 42 (2002), 490–507. doi:10.1023/A:1021973025166
- [Petráš(2011)] Ivo Petráš. 2011. *Fractional-order nonlinear systems: modeling, analysis and simulation*. Springer Science & Business Media. doi:10.1007/978-3-642-18101-6
- [Podlubny(1998)] Igor Podlubny. 1998. *Fractional differential equations: an introduction to fractional derivatives, fractional differential equations, to methods of their solution and some of their applications*. elsevier.
- [Rackauckas and Nie(2017)] Christopher Rackauckas and Qing Nie. 2017. Differentialequations.jl—a performant and feature-rich ecosystem for solving differential equations in julia. *Journal of open research software* 5, 1 (2017), 15–15.

[Westerlund(1991)] Svante Westerlund. 1991. Dead matter has memory! *Physica scripta* 43, 2 (1991), 174.

[Westerlund and Ekstam(2002)] Svante Westerlund and Lars Ekstam. 2002. Capacitor theory. *IEEE Transactions on Dielectrics and Electrical Insulation* 1, 5 (2002), 826–839.